ICCAS 2026 Tutorial

Model Predictive Control as Online Policy Improvement

A Reinforcement Learning Perspective

Kyunghwan Choi

Assistant Professor, KAIST





October 27, 2026 14:35–15:50 (JST)

Tutorial materials and resources

Download the slides and follow along.

Tutorial materials



Slalom Challenge



Textbook



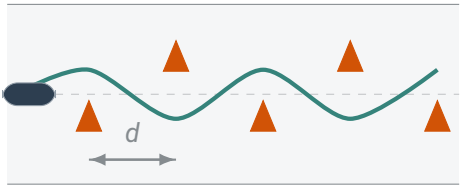
Slalom as an optimal control problem



Pass alternating gates as quickly as possible.

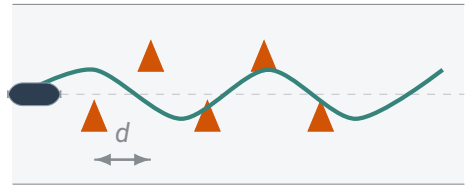
A learned policy in a changed environment

Training environment



A policy learned for this gate spacing.

Changed environment



Closer gates require more frequent turns at the same speed.

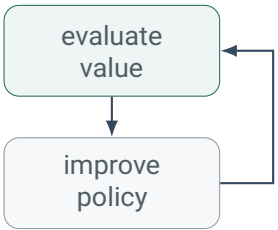
Can we improve the action without retraining the policy?

Offline policy computation and online planning

Offline policy computation

Compute a value function and a policy.

Policy iteration and approximate PI.

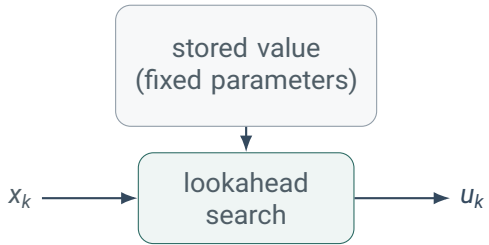


Store a value function and a policy.

Online planning

Use the offline-computed value.

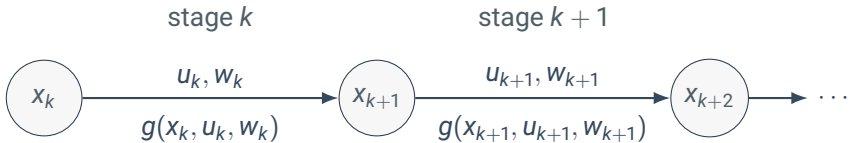
Keep it fixed and **search for an action now**.



Seek **online policy improvement** during control.

Policy Iteration for Optimal Control

Stationary discounted infinite-horizon control



System (environment)

$$
\begin{aligned}
x_{k+1} &= f(x_k, u_k, w_k), \\
x_k &\in \mathcal{X}, \quad u_k \in \mathcal{U}(x_k), \\
w_k &\sim \mathrm{P}^w(\cdot \mid x_k, u_k).
\end{aligned}
$$

Stationary policy

$$u_k = \mu(x_k), \qquad \mu(x) \in \mathcal{U}(x).$$

Value function Expected discounted cost-to-go.

$$J_\mu(x) = \mathbb{E}^\mu \left[\sum_{k=0}^{\infty} \alpha^k g(x_k, \mu(x_k), w_k) \,\middle|\, x_0 = x \right], \quad 0 < \alpha < 1.$$

Optimal value and optimal policy

Assumption 2.3. Finite states and nonempty finite admissible-control sets; bounded stage costs.

Minimize total discounted cost

Optimal value function

$$J^*(x) = \min_{\mu} J_\mu(x), \quad x \in \mathcal{X}$$

Optimal policy

$$\mu^*(x) \in \mathcal{U}(x)$$
$$J_{\mu^*}(x) = J^*(x), \quad \text{for every } x \in \mathcal{X}$$

Current cost and remaining value

$$\sum_{t=0}^{\infty} \alpha^t g_t = g_0 + \alpha(g_1 + \alpha g_2 + \cdots).$$



A fixed policy

The tail follows μ from x_1.

Its remaining expected cost is $J_\mu(x_1)$.

Current cost and remaining value

$$\sum_{t=0}^{\infty} \alpha^t g_t = g_0 + \alpha(g_1 + \alpha g_2 + \cdots).$$



An optimal policy

The tail is optimal from x_1.

Its remaining expected cost is $J^*(x_1)$.

This is the **principle of optimality**.

Bellman equations: evaluation and optimality

Fixed-policy Bellman equation

Fixed policy: evaluate its prescribed action.

$$J_\mu(x) = \mathbb{E}[g(x, \mu(x), w) + \alpha J_\mu(f(x, \mu(x), w)) \,|\, x] \,.$$

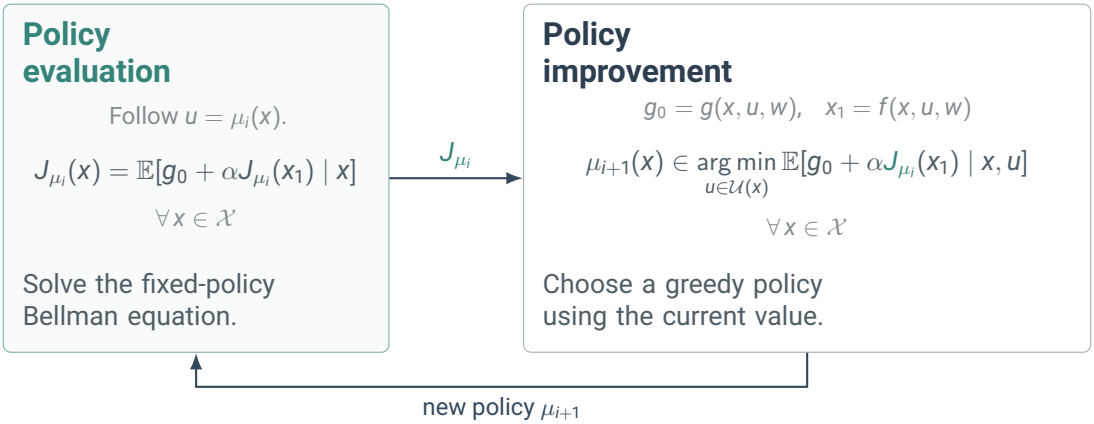
Bellman optimality equation

Optimal policy: minimize over admissible actions.

$$J^*(x) = \min_{u \in \mathcal{U}(x)} \mathbb{E}[g(x, u, w) + \alpha J^*(f(x, u, w)) \,|\, x, u] \,.$$

- Evaluation finds the value J_μ of a given policy.
- Optimality determines J^* and its minimizing action.

Policy iteration for optimal control



Policy improvement and finite termination

Proposition 2.5: exact policy improvement

Under Assumption 2.3, exact evaluation and greedy improvement give

$$J_{\mu_{i+1}}(x) \le J_{\mu_i}(x) \qquad \text{for every } x.$$

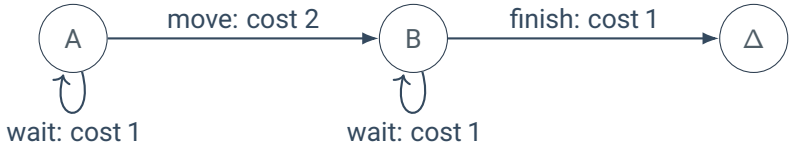
Unless the current policy is already greedy, cost falls at some state.

Only finitely many deterministic stationary policies are possible.

Stop when the policy is greedy with respect to its own value.

Exact PI reaches an optimal policy in finitely many improvements.

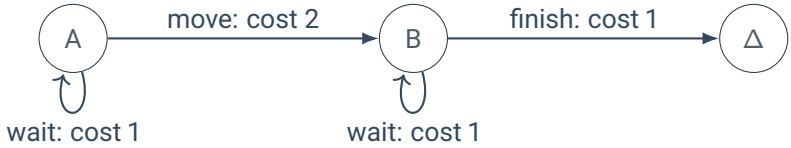
A small policy-iteration example



$\alpha = 0.9$, and $J(\Delta) = 0$.

Iteration	$\mu_i(\mathrm{A})$	$\mu_i(\mathrm{B})$	$J_{\mu_i}(\mathrm{A})$	$J_{\mu_i}(\mathrm{B})$
0	wait	wait	10	10

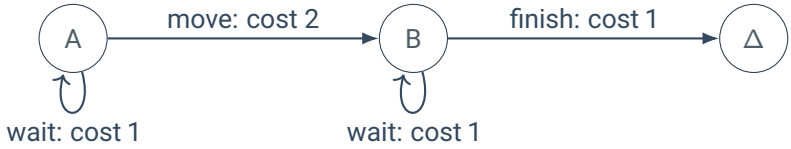
A small policy-iteration example



$\alpha = 0.9$, and $J(\Delta) = 0$.

Iteration	$\mu_i(\mathrm{A})$	$\mu_i(\mathrm{B})$	$J_{\mu_i}(\mathrm{A})$	$J_{\mu_i}(\mathrm{B})$
0	wait	wait	10	10
1	wait	finish	10	1

A small policy-iteration example



$\alpha = 0.9$, and $J(\Delta) = 0$.

Iteration	$\mu_i(\mathrm{A})$	$\mu_i(\mathrm{B})$	$J_{\mu_i}(\mathrm{A})$	$J_{\mu_i}(\mathrm{B})$
0	wait	wait	10	10
1	wait	finish	10	1
2	move	finish	2.9	1

From a small table to continuous control

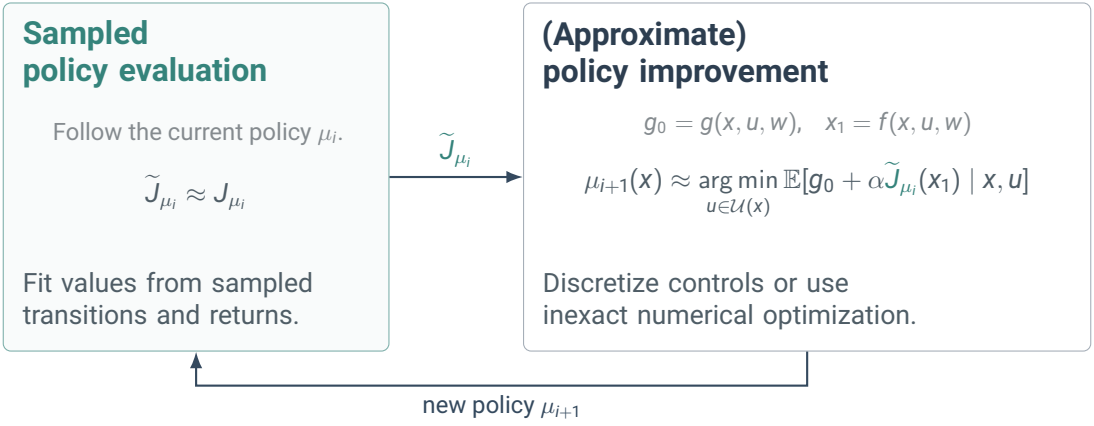
	Small example	Slalom interface
State	$x \in \{\mathrm{A}, \mathrm{B}, \Delta\}$ 1 variable, 3 discrete values	$x^{\mathrm{meas}} \in \mathbb{R}^7$ 7 continuous measured channels
Control	$u \in \{0, 1\}$ 1 variable, 2 choices	$u = (\delta^{\mathrm{req}}, a^{\mathrm{req}}) \in \mathbb{R}^2$ 2 continuous control inputs

- Evaluation over a large continuous state space.
- Improvement requires an optimization over admissible controls.

Assume a prediction model is available.

Approximate Policy Iteration

Approximate policy iteration



Updates use sampled states; function approximation generalizes beyond them.

Monte Carlo and TD evaluation



Monte Carlo

Complete return

$$y_k = \sum_{t=0}^{N-1} \alpha^t g_{k+t}$$

Wait until the episode ends.
Zero-cost terminal after N transitions.

Temporal difference

TD(0): one-step bootstrap

$$y_k = g_k + \alpha \widetilde{J}_\mu(x_{k+1})$$

Update after one transition.
Bootstrap from the current value estimate.

Same value update, different targets

$$\widetilde{J}_\mu(x_k) \leftarrow \widetilde{J}_\mu(x_k) + \eta_k[y_k - \widetilde{J}_\mu(x_k)].$$

A parametric value function is a critic

Represent future cost

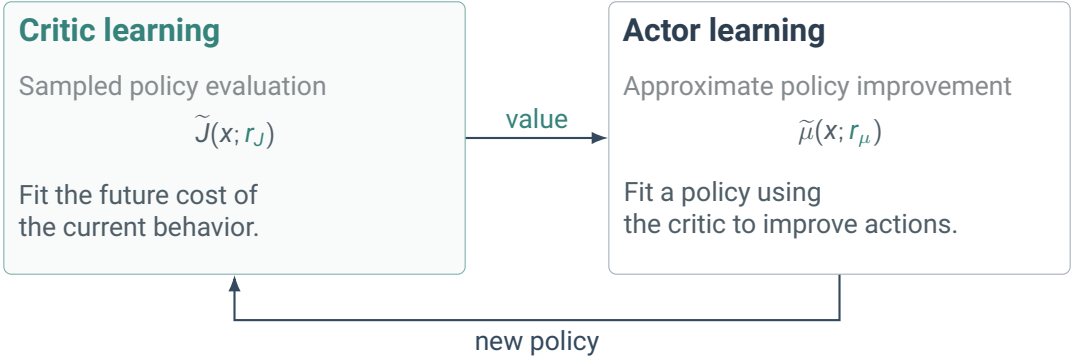
$$\widetilde{J}(x; r_J), \qquad \text{for example } \widetilde{J}(x; r_J) = \phi(x)^{\mathsf{T}} r_J.$$

Critic learning: update the value parameters

$$r_{J,k+1} = r_{J,k} + \eta_k [y_k - \widetilde{J}(x_k; r_{J,k})] \nabla_{r_J} \widetilde{J}(x_k; r_{J,k}).$$

- MC return: the target is independent of critic weights; this equals the full gradient.
- TD target: ignore its dependence on critic weights, giving a **semi-gradient**.

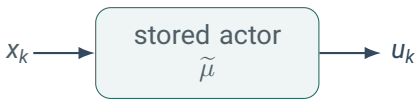
Actor–critic as approximate policy iteration



Actor–critic is one implementation. Numerical action minimization can replace the actor.

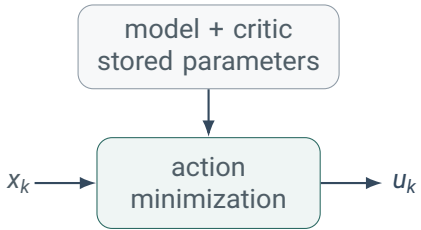
During control: actor execution or online planning

1. Evaluate the stored actor



Fast action evaluation.
No new policy-improvement search.

2. Search using the stored critic



Repeat improvement **at every step**.
This is **online planning**.

Online Planning with Multistep Lookahead

Lookahead with the exact optimal value

Assumption 4.1. Bounded costs and measurable values; well-defined Bellman expectations preserving this class; admissible measurable minimizers.

Exact model: $x_0 = x$, $x_{t+1} = f(x_t, \mu_t(x_t), w_t)$; admissible feedback decisions.

$\ell = 1$ 

$$\min_{\mu_0} \quad \mathbb{E}[g_0 + \alpha J^*(x_1)] \qquad = J^*(x)$$

$\ell = 2$ 

$$\min_{\mu_0,\mu_1} \quad \mathbb{E}[g_0 + \alpha g_1 + \alpha^2 J^*(x_2)] \qquad = J^*(x)$$

ℓ 

$$\min_{\mu_0,\ldots,\mu_{\ell-1}} \quad \mathbb{E}\left[\sum_{t=0}^{\ell-1} \alpha^t g_t + \alpha^\ell J^*(x_\ell)\right] \qquad = J^*(x)$$

Every depth gives optimal actions and value.

Lookahead with a stored approximate value

Fixed critic $\widetilde{J}$ Online planning searches from the current state.

$\widetilde{J}^{(\ell)}(x)$: minimum search cost at depth ℓ.

$\ell = 1$ 

$$\min_{\mu_0} \quad \mathbb{E}[g_0 + \alpha \widetilde{J}(x_1)] \qquad = \widetilde{J}^{(1)}(x)$$

$\ell = 2$ 

$$\min_{\mu_0,\mu_1} \quad \mathbb{E}[g_0 + \alpha g_1 + \alpha^2 \widetilde{J}(x_2)] \qquad = \widetilde{J}^{(2)}(x)$$

ℓ 

$$\min_{\mu_0,\ldots,\mu_{\ell-1}} \quad \mathbb{E}\left[\sum_{t=0}^{\ell-1} \alpha^t g_t + \alpha^\ell \widetilde{J}(x_\ell)\right] \qquad = \widetilde{J}^{(\ell)}(x)$$

Let $(\widehat{\mu}_0^{(\ell)}, \ldots, \widehat{\mu}_{\ell-1}^{(\ell)})$ attain the minimum. Apply its first control: $\widetilde{\mu}^{(\ell)}(x) := \widehat{\mu}_0^{(\ell)}(x)$.

With an exact model, which depth would you trust?

Lookahead attenuates terminal-value error

Proposition 4.3: exact lookahead under Assumption 4.1

Suppose $\|\widetilde{J} - J^*\|_\infty \le \varepsilon$.

One step

$$\|J_{\tilde{\mu}^{(1)}} - J^*\|_\infty \le \frac{2\alpha}{1-\alpha}\varepsilon.$$

ℓ steps

$$\|J_{\tilde{\mu}^{(\ell)}} - J^*\|_\infty \le \frac{2\alpha^\ell}{1-\alpha}\varepsilon.$$

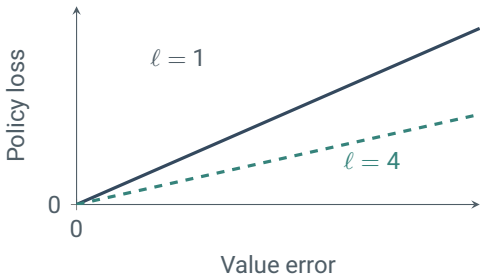
Longer exact lookahead reduces the influence of terminal error.

- $\ell = 1$ is the greedy policy induced by the offline-computed value $\widetilde{J}$.
- For the same $\widetilde{J}$ and α, $\ell > 1$ gives a smaller worst-case loss bound.

This motivates online policy improvement. Actual cost need not fall at every depth.

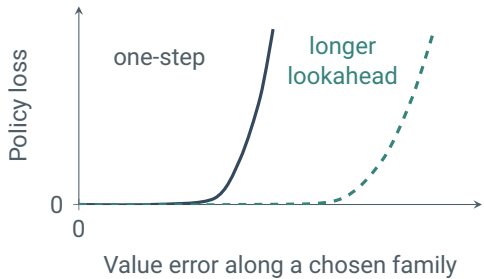
Global bounds and local performance

Global worst-case envelope



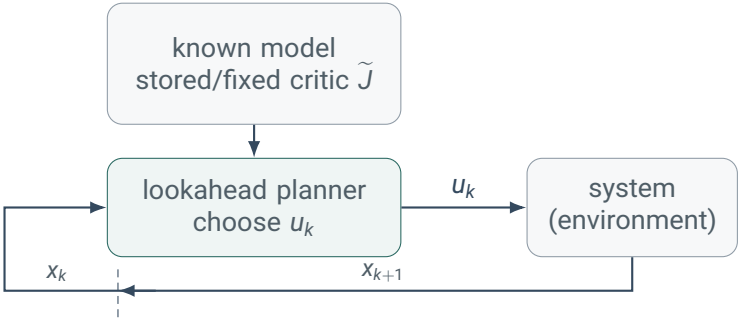
Upper bounds for $\alpha = 0.8$.

Local behavior near a good value



Qualitative behavior, not a universal bound.

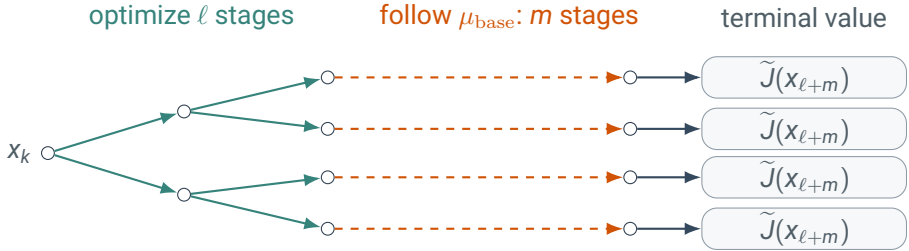
Online planning in a receding horizon



Apply the first control; solve again at the next state.

Receding-horizon execution gives an **MPC structure**.

Truncated rollout before terminal evaluation

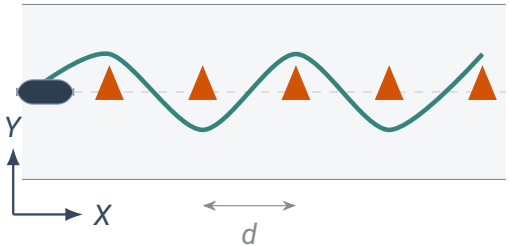


$$\min_{\mu_0,\ldots,\mu_{\ell-1}} \mathbb{E}\left[\sum_{t=0}^{\ell-1} \alpha^t g_t + \sum_{t=\ell}^{\ell+m-1} \alpha^t g(x_t, \mu_{\text{base}}(x_t), w_t) + \alpha^{\ell+m}\widetilde{J}(x_{\ell+m})\right].$$

- Base-policy simulation reduces the number of optimized decisions.
- Terminal errors enter with $\alpha^{\ell+m}$; base-policy suboptimality remains.

Nominal slalom: problem and learning setup

Nominal environment: $d = 15$ m



State and control

$$x_k = (X_k, Y_k, \psi_k, v_{x,k}, v_{y,k}, r_k, \delta_k)^\top$$

$$u_k = (\sigma_k, a_k^{\mathrm{req}})^\top$$

Steering command and requested acceleration.

Discounted infinite-horizon control

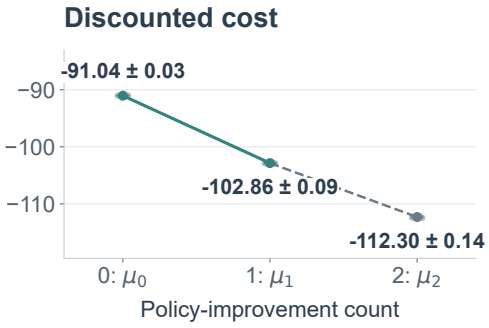
$$g_k = -\underbrace{(X_{k+1} - X_k)}_{\text{forward progress}} + c_{\mathrm{risk},k}.$$

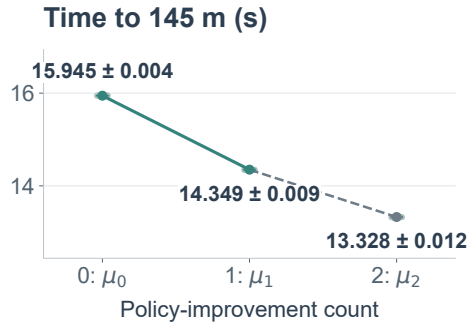
First physical failure: cost 200, followed by absorption.

Approximate PI: roll out μ_i for evaluation; improve and store μ_{i+1}.

Approximate PI in the nominal environment

Nominal gate spacing $d = 15$ m. Five paired starts, mean $\pm$ SD.

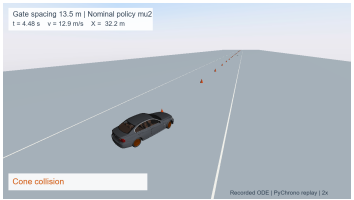
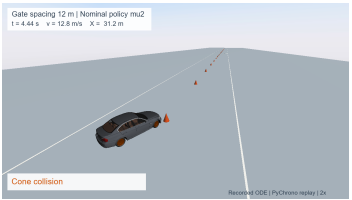




Two PI updates reduce cost and arrival time.

The same policy in three environments

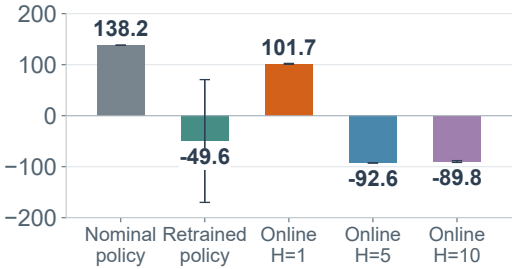
Stored μ_2 after the second PI update; no further learning.

Nominal: 15 m	Changed: 13.5 m	Changed: 12 m
		
13.33 s	**Collision**	**Collision**
5/5 reach 145 m	0/5 reach 145 m	0/5 reach 145 m

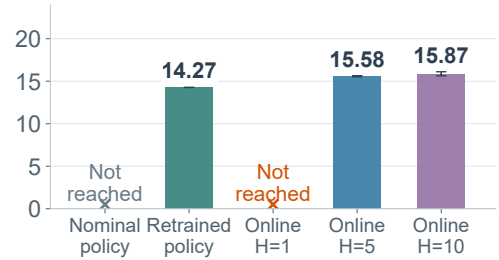
Transfer at 13.5 m: retrain or plan online

Online planning: frozen nominal value, $H = 1, 5, 10$. Retraining: four PI updates.

Discounted cost · lower is better



Time to 145 m · seconds



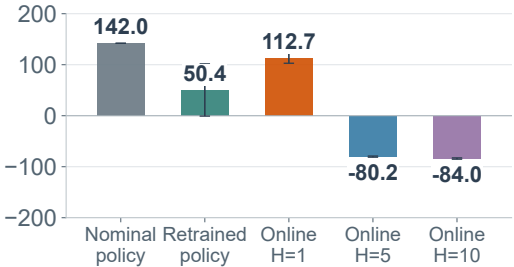
	Nominal policy	Retrained policy	Online H=1	Online H=5	Online H=10
Arrivals	0/5	4/5	0/5	5/5	5/5
Physical failures	5/5	1/5	5/5	0/5	0/5

Online planning with $H = 5$ or 10 restores progress.

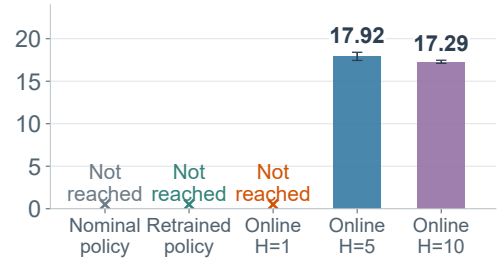
Transfer at 12 m: lookahead recovers progress

Online planning uses the same frozen nominal value: $H = 1, 5, 10$.

Discounted cost · lower is better



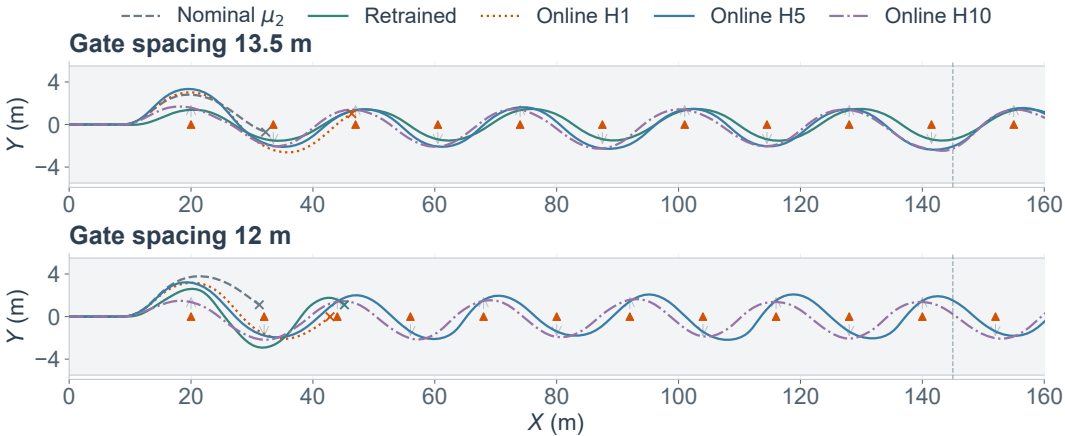
Time to 145 m · seconds



	Nominal policy	Retrained policy	Online H=1	Online H=5	Online H=10
Arrivals	0/5	0/5	0/5	5/5	5/5
Physical failures	5/5	5/5	5/5	0/5	0/5

Online planning with $H = 5$ or 10 recovers progress without retraining.

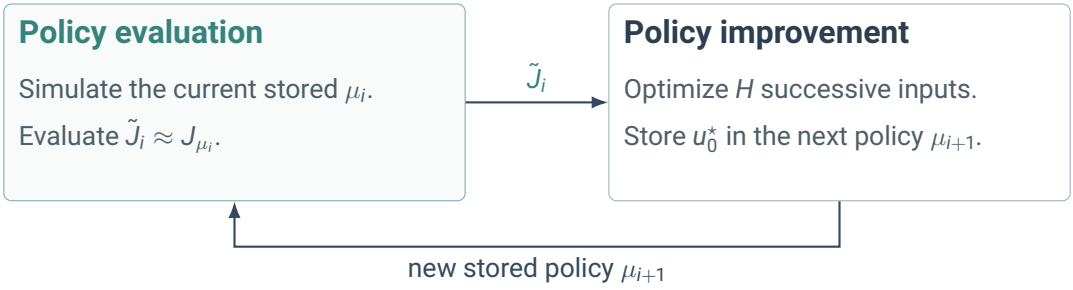
How the policies move through the changed course



Online planning restores progress with a fixed critic.

Multistep lookahead inside approximate PI

During training: same evaluation, deeper policy improvement.

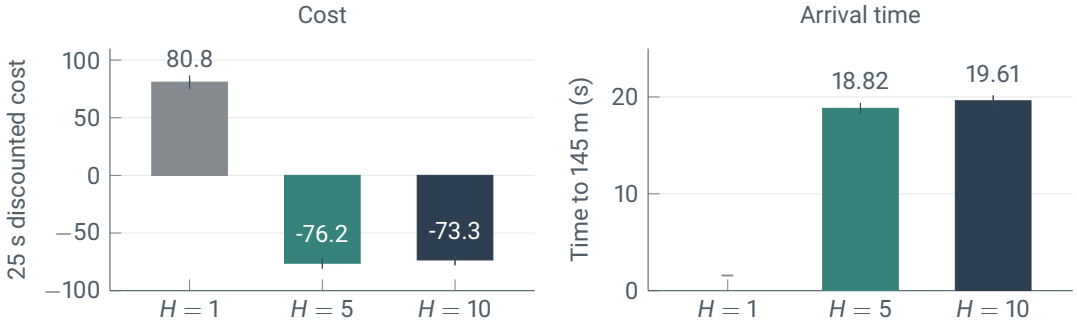


$$(u_0^\star, \ldots, u_{H-1}^\star) \in \underset{u_0,\ldots,u_{H-1}}{\arg\min} \left[\sum_{t=0}^{H-1} \alpha^t g(x_t, u_t) + \alpha^H \tilde{J}_i(x_H) \right].$$

Execution uses the stored policy, without a planner.

Multistep policy improvement at 12 m

Four additional PI updates; H is the lookahead depth **during training**.



Safe arrivals: $H = 1$: **0/10** $H = 5$: **10/10** $H = 10$: **10/10**

The stored policies learned with $H = 5$ and 10 complete the course.

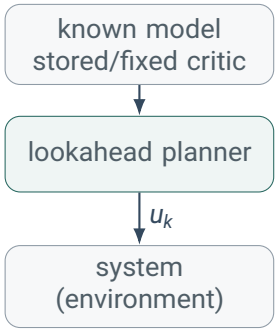
Stored-policy execution, without an online planner. Same 10 test starts; mean ± SD.

Online Planning During Reinforcement Learning

The same planner, two roles

After training: during control

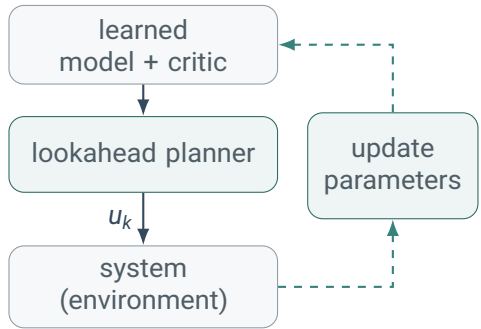
Improve the current action



Use fixed parameters to choose the action.

During reinforcement learning

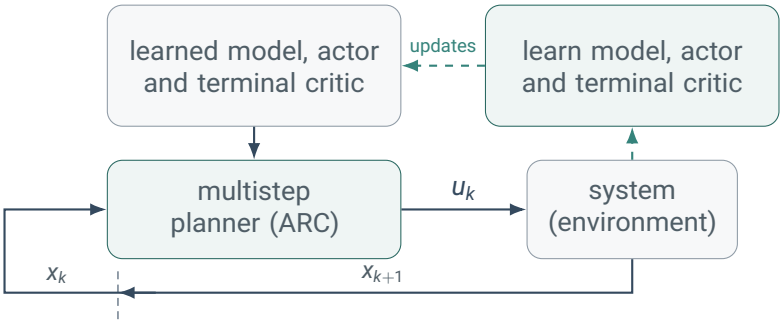
Improve the data used for learning



Use planned actions to collect learning data.

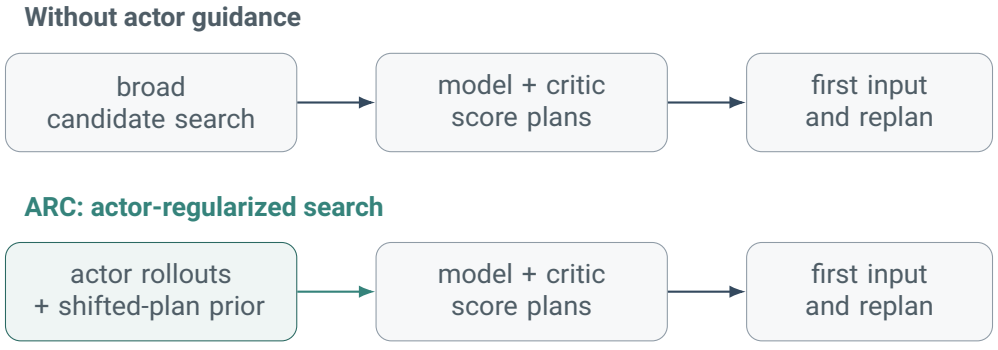
LOOP: learning with online planning

Learning Off-Policy with Online Planning (LOOP) [1]



Planning selects the actions used to collect training data.

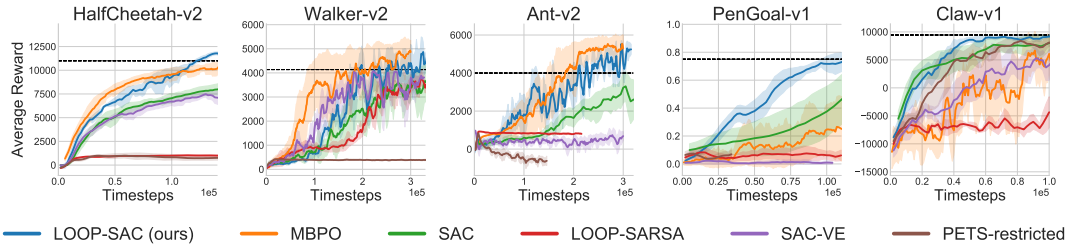
ARC adds actor guidance inside the planner



Actor guidance changes how candidate trajectories are proposed.

ARC is LOOP's search subroutine. It can also run after training with fixed functions. [1]

Online planning changes the learning process



Planning changes the **experience used for learning**. [1]

Dashed lines: SAC performance at one million timesteps.

Further examples of planning and learning

POLO [2]

Local trajectory optimization with a learned global value function.

MPC / value blending [3]

Blend learned Q estimates with model-based returns across depths.

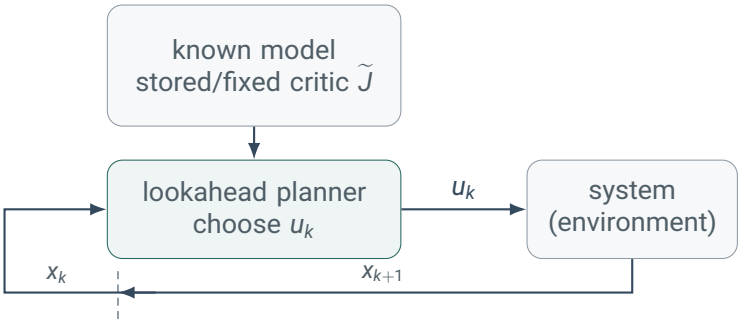
Further reading

TD-MPC / TD-MPC2: learned latent models and values for planning.

AC4MPC: actor–critic guidance for MPC. [4–6]

Can the critic keep learning during control?

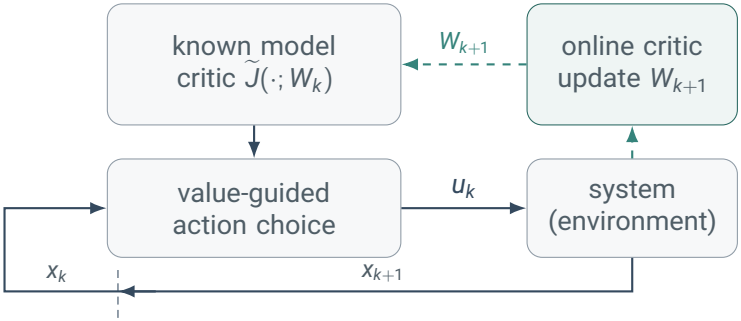
RL training → **during control**



What if we update this critic in real time?

Online Critic Learning for Optimal Control

Online critic learning during control



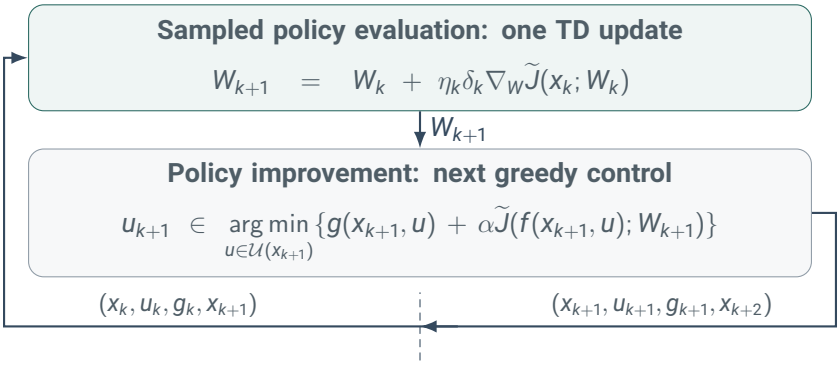
Here $W_k = r_{J,k}$ denotes the critic parameters.

Now update the critic to improve subsequent actions.

TD-based online critic learning

Approximate PI during control. Deterministic model: $x_{k+1} = f(x_k, u_k)$.

$$y_k = g_k + \alpha \widetilde{J}(x_{k+1}; W_k), \qquad \text{TD error: } \delta_k = y_k - \widetilde{J}(x_k; W_k).$$



Control-oriented learning laws typically include **Lyapunov-based stability analysis**.

What does a small TD error establish?

Sampled policy evaluation — Fixed-policy Bellman equation

$$J_\mu(x) = g(x, \mu(x)) + \alpha J_\mu(f(x, \mu(x))).$$

$$\varepsilon_\mu(x) = g(x, \mu(x)) + \alpha \widetilde{J}(f(x, \mu(x))) - \widetilde{J}(x).$$

TD update: $\delta_k = \varepsilon_{\mu_k}(x_k)$ for the current critic.

Minimize over admissible actions — Bellman optimality equation

$$J^*(x) = \min_{u \in \mathcal{U}(x)} \{g(x, u) + \alpha J^*(f(x, u))\}.$$

$$\varepsilon^*(x) = \min_{u \in \mathcal{U}(x)} \{g(x, u) + \alpha \widetilde{J}(f(x, u))\} - \widetilde{J}(x).$$

Small sampled TD errors do not establish Bellman optimality.

Proposed: Bellman optimality as critic learning

$$J^*(x) = \min_{u \in \mathcal{U}(x)} \{g(x,u) + \alpha J^*(f(x,u))\}.$$

$$\Downarrow$$

Approximate J by $\widetilde{J}(x;W) = W^{\mathsf{T}}\phi(x)$ and retain both operations.

Proposed sampled constrained problem

$$\min_{u \in \mathcal{U}(x),\, W} \quad F(x,u;W) := g(x,u) + \alpha \widetilde{J}(f(x,u);W)$$

$$\text{subject to} \quad F(x,u;W) - \widetilde{J}(x;W) = 0.$$

Lee and Choi, 2026. [7]

TD and proposed updates at the sampled state

Linear critic: $\widetilde{J}(x;W) = W^{\mathsf{T}}\phi(x)$; $\quad \varepsilon_k = F(x_k, u_k; W_k) - \widetilde{J}(x_k; W_k) = \delta_k$.

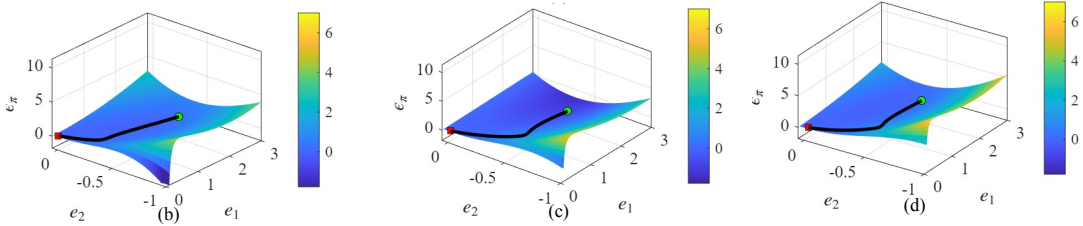
	TD-based learning	Proposed
Target at x_k	At the sampled action $\delta_k \longrightarrow 0$	$\varepsilon^*(x_k; W_k) \longrightarrow 0$
Weights	$W_{k+1} = W_k + \eta_k \varepsilon_k \phi(x_k)$	$W_{k+1} = W_k - \eta_W \nabla_W \mathcal{L}_k^{\mathrm{aug}}$
Multiplier	No equality multiplier	$\lambda_{k+1} = \lambda_k + \rho(\varepsilon_k - \sigma_\lambda \lambda_k)$

$\mathcal{L}_k^{\mathrm{aug}}$: augmented Lagrangian; $\quad \lambda_k$: Bellman-equality multiplier.

Bellman residuals across the evaluation region

Bellman residual $\varepsilon_\mu(x)$: at the applied action

Proposed **TD(0)** **Normalized ADP**



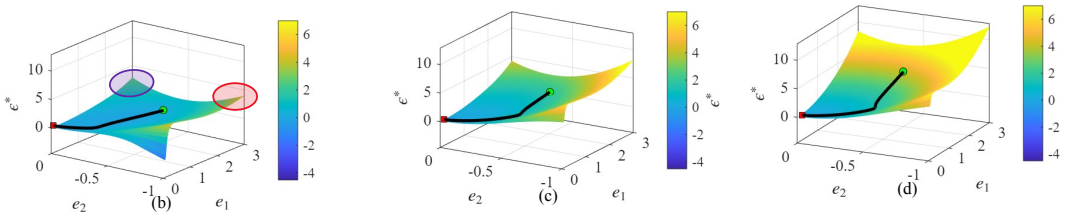
Small Bellman residuals **along the visited trajectories**.

Does this also imply Bellman optimality away from them?

Bellman residuals across the evaluation region

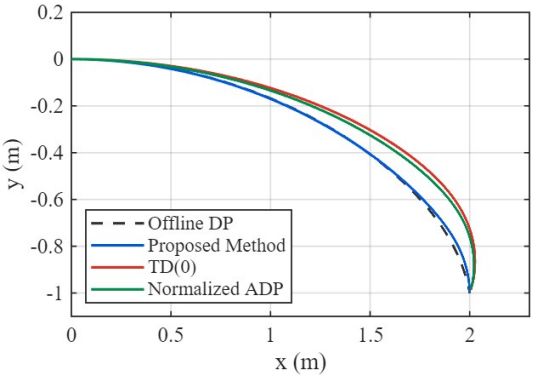
Bellman optimality residual $\varepsilon^*(x)$: minimize over actions

Proposed | **TD(0)** | **Normalized ADP**



Proposed has the **smallest accumulated optimality residual** among the three online methods on the evaluated grid.

Trajectory tracking after critic learning



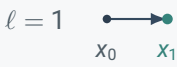
Learned control follows the offline DP reference more closely.

Adapt the critic or search deeper?

Online critic learning

$$W_k \longrightarrow W_{k+1}$$

Update the evaluator during control.



One-step improvement

$$\min_{u \in \mathcal{U}(x)} \mathbb{E}[g_0 + \alpha \widetilde{J}(x_1; W_k) \mid x, u].$$

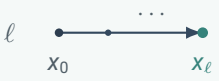
Adaptation

Learning transients / stability

Frozen critic + lookahead

$$\widetilde{J} \quad \text{fixed}$$

Recompute a deeper action search.



ℓ-step improvement

$$\min_{\mu_0, \ldots, \mu_{\ell-1}} \mathbb{E}\left[\sum_{t=0}^{\ell-1} \alpha^t g_t + \alpha^\ell \widetilde{J}(x_\ell)\right].$$

Replanning

Model error / search time

Key takeaways

A value function compresses the future.

Policy evaluation estimates that future cost; improvement compares actions.

Multistep lookahead can improve decisions online.

Stored parameters can stay fixed while receding-horizon planning selects new inputs.

Online critic learning adapts the value function.

The proposed update targets Bellman optimality consistency at sampled states.

Questions and resources

Thank you

Tutorial materials



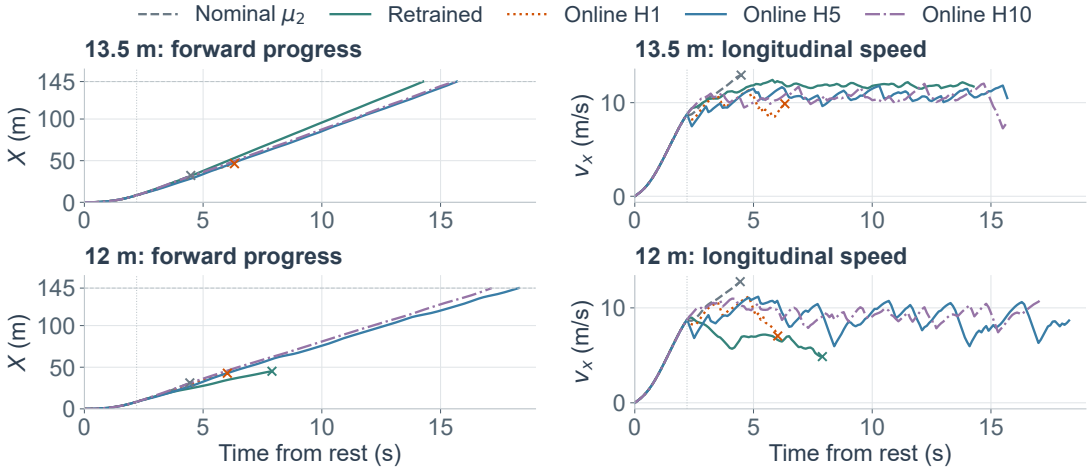
Slalom Challenge



Textbook



Backup: speed and progress in the changed course



Each trace ends at 145 m or physical failure. ×: failure before arrival.
Launch from rest is included; dotted line: policy activation.

References: planning and value learning

[1] H. Sikchi, W. Zhou, and D. Held. **"Learning Off-Policy with Online Planning".** In: *Proceedings of the 5th Conference on Robot Learning*. Vol. 164. Proceedings of Machine Learning Research. 2022, pp. 1622–1633. URL: https://proceedings.mlr.press/v164/sikchi22a.html.

[2] K. Lowrey, A. Rajeswaran, S. Kakade, E. Todorov, and I. Mordatch. **"Plan Online, Learn Offline: Efficient Learning and Exploration via Model-Based Control".** In: *International Conference on Learning Representations*. 2019. arXiv: 1811.01848.

[3] M. Bhardwaj, S. Choudhury, and B. Boots. **"Blending MPC & Value Function Approximation for Efficient Reinforcement Learning".** In: *International Conference on Learning Representations*. 2021. arXiv: 2012.05909.

[4] N. Hansen, H. Su, and X. Wang. **"Temporal Difference Learning for Model Predictive Control".** In: *Proceedings of the 39th International Conference on Machine Learning*. Vol. 162. Proceedings of Machine Learning Research. 2022, pp. 8387–8406. URL: https://proceedings.mlr.press/v162/hansen22a.html.

[5] N. Hansen, H. Su, and X. Wang. **"TD-MPC2: Scalable, Robust World Models for Continuous Control".** In: *International Conference on Learning Representations*. 2024. arXiv: 2310.16828.

References: critic learning and analysis

[6] R. Reiter, A. Ghezzi, K. Baumgärtner, J. Hoffmann, R. D. McAllister, and M. Diehl. **"AC4MPC: Actor-Critic Reinforcement Learning for Guiding Model Predictive Control".** In: *IEEE Transactions on Control Systems Technology* (2025). Accepted author manuscript. DOI: 10.1109/TCST.2025.3620521.

[7] H. Lee and K. Choi. **"From Bellman Consistency toward Bellman Optimality: Constrained Online Critic Learning for Nonlinear Optimal Control".** Manuscript under review. 2026. URL: https://kaist-mic-lab.github.io/static/pub/2026-from-bellman.pdf.

[8] D. P. Bertsekas. ***Model Predictive Control and Reinforcement Learning: A Unified Framework Based on Dynamic Programming***. 2024. arXiv: 2406.00592.